

Appendix Feature IDs

Proposal

Mini-UID: A simple, alphanumeric UID

- 4-characters long
- Characters can include 0-9 and a-f
- First character however, must be a number
- examples:
 - `%[control:slider:4e97]`
 - `%[task:1ba0]`
 - `%[output:89d3]`
 - 255f, 0660, 6cb6, 9819, 192d, 6826, 48bb, 4c73

Rationale

The basic integer ID solution heavily prioritized readability over robustness, and so this solution proposes to move firmly in the other direction.

Risk of collision: Unlike the original proposal (*basic incrementor*) which is likely to have a collision when each version of the file introduces **1** new feature of a given type, this proposal makes a collision unlikely to happen until the point that each file introduces **>75** instances of a feature of a given type. (And the proposed design strives to manage a decent level of readability along with it!)

If a collision does occur, the user needs to intervene and change the ID of that feature – or it would ultimately be handled as a case of corruption.

Use-case:

- This would only realistically occur when 2 different versions of a file each have >75 new outputs added to it
 - AND the user then wants to merge both sets of outputs into a single file
- Even in this case, we would expect only a single collision
- This is a massive improvement over the existing solution of an integer (*in which case there would likely be 75 conflicts*)

Therefore, I would propose that this design should be sufficient, but recognize that it could incur ID conflicts for outputs in some rare (and low-priority) workflows.

Alternate solutions

If others propose that we would prefer to further prioritize *Risk of Collision* over *Readability*, we can further extend the character set or length of the ID using the alternatives below.

Alternate 1

Add an extra character to ID length for more robustness.

- `%[control:slider:6c523]`
- `%[task:96820]`

- `%[output:10c1b]`
- `07e28, 561cf, 9a26e, 35bce, 55211, 123e3, 331ff, 1cf13`

Risk of collision: Both files would need to introduce >300 instances of a given feature type for a single collision to be likely

This approach increases our safety, but slightly reduces readability by introducing a longer ID.

Alternate 2

Increase the character set from "a-f" to "a-z"

- `%[control:slider:8ccn]`
- `%[task:5foy]`
- `%[output:19fw]`
- `6fcu, 2kvs, 4pju, 1y9c, 0tve, 9jcq, 85gy, fnpz`

Risk of collision: Both files would need to introduce >300 instances of a given feature type for a single collision to be likely

This approach increases our safety, but slightly reduces readability by introducing less "neat" characters than those used by UUID.

Alternate 3

Continue to add to the ID length or character set, moving closer to a UUID and further away from a friendly or readable ID.

See table below for more considerations.

Alternate 4

Use a basic incrementing integer.

- `%[control:slider:3]`
- `%[task:1]`
- `%[output:10]`
- `1, 2, 3, 4, 5, 6, ...`

Risk of collision: Both files would need to introduce **1** instance of a given feature type for a collision to be likely

This approach is very simple and understandable, but the primary issue is the frequency of conflicts – it could be quite common for some workflows. The IDs could also become unordered over time as the file is edited, and this could be confusing to the user.

Investigation

Benchmarks and resources

UUIDs only use the letters "a-f" (and some offshoot implementations also include A-F), but have a length of 32 characters:

- <https://stackoverflow.com/questions/8258480/type-of-character-generated-by-uuid>

According to this answer below, a 4-digit alphanumeric UID (using a-z and A-Z) is considered safe and unlikely to have an ID collision until 1300 IDs have been generated (without checking):

- <https://stackoverflow.com/questions/6248666/how-to-generate-short-uid-like-ax4j9z-in-js>

Why start with a number?

Due to syntactical constraints of `##element:type:id`, where type and ID are both options pieces, we distinguish by establishing that Element and Type must be valid ML identifiers, while the ID must not be.

We considered other syntaxes like `##element:type[id]` that would bypass the problem. But these were flagged as less simple, and bringing false semantic parallels to the ML coding language.

Length vs Character set

Want to assess the trade-off between feature set and ID length. We want to identify a combination which provides sufficient permutations along with a high level of readability.

A few initial observations

- only including "a-f" characters greatly seems to increase the readability and cleanliness of the design
- the downside is this significantly reduces the character set compared with "a-z" (36 → 16 character set)
- UUID only uses "a-f", but they have very long IDs
- a larger ID length tends to increase N far more than a larger character set does

In the table below, we look at several different types of ID configurations, provide a few samples, and identify the "safe" number of IDs without a likely collision. Note that the numbers are rounded/approximate, and that the number of "safe" IDs (without checking) can be approximated using " $\sqrt{N}$ " [\[source\]](#).

✓ Table without a pre-pended number...

Character set	Length of 3	Length of 4	Length of 5
0-9, a-f	N: 3,000	N: 40,000	N: 500,000
<i>16 characters</i>	Safe: 55	Safe: 200	Safe: 700
	1d1 // f89	40d6 // 6d44	c9e92 // 2426c

0-9, a-z <i>36 characters</i>	N: 42,000 Safe: 200 n8h // r5l	N: 1.4 million Safe: 1000 ijir // lgxi	N: 45 million Safe: 6500 u4gtg // s22hf
0-9, a-z, A-Z <i>62 characters</i>	N: 200,000 Safe: 450 lKz // Jf1	N: 13 million Safe: 3500 oLpf // WNf6	N: 750 million Safe: 27k 8UgaD // q2RyT

Character set	Length: 1+3	Length: 1+4	Length: 1+5
0-9, a-f <i>16 characters starts with a digit</i>	N: 20k Safe: 150 41d1 // 2f89	N: 350k Safe: 600 540d6 // 76d44	N: 4.5m Safe: 2k 4c9e92 // 82426c
0-9, a-z 0-9, A-Z <i>36 characters starts with a digit</i>	N: 350k Safe: 600 5x8p // 4olw 5X8P // 4OLW	N: 10m Safe: 3k 7ij0r // 1lgxi 7IJ0R // 1LGXI	N: 400m Safe: 20k 0u4gtg // 3s22hf 0U4GTG // 3S22HF
0-9, a-z, A-Z <i>62 characters starts with a digit</i>	N: 1.8m Safe: 1.5k 2lKz // 7Jf1	N: 100m Safe: 10k 8oLpf // 9WNf6	N: 6.7b Safe: 80k 78UgaD // 4q2RyT

Risk of Collision

What does the "**safe**" number really mean? And how big does it really need to be?

Summary: Each version of the file needs to introduce **Safe/2 new features (of a given type)** in order have a >50% chance of a single collision.

✓ Click here to expand...

- The "safe" number is the number of IDs that can be randomly generated with < 50% chance of collision.
- Say our Safe value = 150 (like in the original proposal)
 - then if we have a file with 75 IDs (10 controls, 5 live tasks, 10 rich text elements needing IDs, and 50 outputs)
 - then say we have to merge it with another version
 - we could imagine that ~72-ish of the IDs would be consistent across the two files (started out as the same file!) (this assumes we persist IDs across multiple saves – like we intend to)
 - so there's **actually only 3*2 = 6 new IDs** which could conflict!

- *it's actually even less than that*, because it's checking 3 against another 3. but not against each other. that seems like hard math. therefore we can call this as 6 random IDs
- in that case, a Safe number of 150 is quite good
- what if there are 50 new outputs in both versions of the file?
 - we don't know how to calculate that, so we'll pretend it's 100 random outputs.
 - still < 50% chance of even a single collision
- what if there are 500 new outputs in both versions?
 - we don't know how to calculate that, so we'll pretend it's 1000 random outputs
 - now it's quite likely to have a few collisions. but it's still not like everything will collide!
 - plus, how were you planning on merging that file anyways?
- It's interesting that outputs are really the only issue here. nothing else should appear in such high quantities. But *merging* of outputs is a low-priority workflow. Suggested to take all from one file, or all from the other file.

How many total permutations do we need?

Even our smallest proposed option leads to ~20,000 possible permutations (for a given feature type / pragma).

This seems like a vastly sufficient number (and to be safe, we could consider increasing ID length if that limit is ever hit).

Therefore this is unimportant to our analysis, and we should focus on Collision Risk & Readability.